Day 61 — A Many-to-One RNN Classifier on Real Short-Text Sentiment

Week 9 · Sequence Data & RNNs I · Complete study guide

Watch: Word Embedding and Word2Vec, Clearly Explained!!! (StatQuest with Josh Starmer) | L15.7 An RNN Sentiment Classifier in PyTorch (Sebastian Raschka)

Script: `day61_many_to_one_sentiment.py` · every number in this guide is copied by program from `day61_run_output.txt`, the real output of that script.

0. What today is about

Days 56–60 trained the RNN on made-up sequences of numbers. Today it reads **real sentences written by real people**: 3,000 one-sentence reviews from Amazon, IMDb and Yelp, each marked positive or negative (Kotzias et al., KDD 2015, the UCI "Sentiment Labelled Sentences" dataset). The job is **many-to-one**: many words go in, one answer comes out.

Three new parts are needed before an RNN can read text, and each is built and checked: a **text pipeline** (words to numbers), an **embedding table** (a learned list of numbers for each word), and **masking** (so the blank filler added to short sentences is ignored). Then comes the honest part: the RNN is compared against a much simpler method that ignores word order, and tested for whether it uses word order at all.

The short version of the result. The RNN works, but on this small dataset it does **not** beat the simple method: the order-blind baseline scores 0.817 on the test sentences; the RNN reading its last state averages 0.737, and reading the average of all its states, 0.797. Scrambling the words of the training sentences did not make the RNN worse. Section 6 shows the evidence.

How to read each section

Every section follows the same seven steps, so you always know where you are:

Step	What it gives you
DEFINITION	What the idea is, in plain words.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
SPECIFICATION	The exact setup of the experiment: data, sizes, settings, what is measured.
Why this test	The question the experiment answers, and why it matters.
IMPLEMENTATION	The real code from the script that produces the result.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.

Key words used today

Word	Plain meaning
Token	One piece of a sentence after cutting it up — usually one word.
Vocabulary	The numbered list of words the model knows.
<unk>	“Unknown”: the stand-in for any word that is not in the vocabulary.
Padding (<pad>)	Blank filler added to short sentences so all sentences have the same length.
Embedding	A short list of numbers that stands for one word. The model learns these numbers.
Mask	A row of 1s and 0s: 1 = real word, 0 = padding.
Many-to-one	Many inputs (the words), one output (the label).
Readout	The rule that turns the RNN's hidden states into one prediction.
Baseline	A simple method used as the bar a fancier method must beat.
Training / validation / test	Three separate piles of sentences: one to learn from, one to decide when to stop, one to give the final honest score.
Overfitting	Memorizing the training examples instead of learning a rule that works on new ones.

1. Turning text into numbers

DEFINITION

Tokenizing means cutting a sentence into pieces called tokens (here: lowercase words, plus “!” and “?”). A **vocabulary** is a numbered list of the words the model knows. **Encoding** replaces each token with its number. **Padding** adds a filler number (0) to the end of short sentences so that every sentence has the same length and they can be stacked into one table.

REAL-WORLD SCENARIO

Think of the contacts list on a phone. You think in names, but the phone only dials numbers, so the list maps each name to a number. A call from someone who is not in the list shows up as “Unknown” — that is exactly what <unk> is for a word the model never learned.

Padding is like a paper form with 40 boxes for your name. “Ali” fills 3 boxes and leaves 37 empty. The empty boxes are not part of the name; they are only there so every form has the same shape.

SPECIFICATION

Data	3,000 sentences: 1,000 each from Amazon, IMDb, Yelp; 1,500 positive, 1,500 negative.
Split	One fixed random split (seed 0): 2,000 training, 400 validation, 600 test.
Tokenizer	Lowercase; keep words, contractions such as *don't*, and ! and ? as tokens. Digits and other punctuation are dropped.
Vocabulary	Built from the **training** sentences only. A word gets its own id if it appears at least twice. Id 0 = <pad>, id 1 = <unk>.
Length	Each sentence is cut to at most 40 tokens and padded with 0 up to 40.
Measured	Sentence lengths, vocabulary size, and how many test tokens are unknown.

Why this test

A network can only multiply and add numbers. Before any learning happens, every decision here — what counts as a word, which words are kept, how long a sentence may be — quietly decides what the model is able to see. The vocabulary is built from training sentences only, because the test sentences must stay unseen: in real use, new reviews will contain words the model has never met.

IMPLEMENTATION

```
def tokenize(sentence):
    return re.findall(r"[a-z]+(?:'[a-z]+)?|[!?]", sentence.lower())
def build_vocab(token_lists, min_count=2):
    counts = Counter(w for toks in token_lists for w in toks)
    itos = ["<pad>", "<unk>"] + sorted(w for w, c in counts.items() if c >= min_count)
    return itos, {w: i for i, w in enumerate(itos)}
def encode(token_lists, stoi, max_len=MAX_LEN):
    ids = np.full((len(token_lists), max_len), PAD, dtype=int)
    lens = np.zeros(len(token_lists), dtype=int)
    for row, toks in enumerate(token_lists):
        seq = [stoi.get(w, UNK) for w in toks][:max_len] or [UNK]
        ids[row, :len(seq)] = seq
        lens[row] = len(seq)
    return ids, lens
```

The subtle part: `stoi.get(w, UNK)` returns 1 for any word that is not in the vocabulary, and `or [UNK]` makes sure a sentence is never empty.

ACTUAL OUTPUT

```
sentences: 3000  (positive 1500, negative 1500)  sources: amazon 1000, imdb 1000, yelp 1000
split: train 2000, validation 400, test 600  (positive fraction 0.501 / 0.517 / 0.483)
tokens per sentence: min 0, median 10, 90th percentile 22, max 73; 20 sentences longer than 40 are
    truncated; 2 have no word tokens and become a single <unk>
distinct words in training sentences: 4053; kept (seen at least twice): 1721; vocabulary size with
    <pad> and <unk>: 1723
test tokens that are <unk>: 14.4%
worked example (training sentence, label 0):
  text   : I have been to very few places to eat that under no circumstances would I ever return
      to, and this tops the list.
  tokens : ['i', 'have', 'been', 'to', 'very', 'few', 'places', 'to', 'eat', 'that', 'under',
      'no', 'circumstances', 'would', 'i', 'ever', 'return', 'to', 'and', 'this', 'tops', 'the',
      'list']
  ids    : [737, 682, 130, 1526, 1606, 554, 1100, 1526, 461, 1485, 1573, 988, 1, 1700, 737, 497,
      1231, 1526, 52, 1507, 1, 1488, 857]  + 17 padding zeros
  decoded: ['i', 'have', 'been', 'to', 'very', 'few', 'places', 'to', 'eat', 'that', 'under',
      'no', '<unk>', 'would', 'i', 'ever', 'return', 'to', 'and', 'this', '<unk>', 'the', 'list']
Actual output from this lesson's run.
```

What it means

The classes are balanced, so guessing gives 0.500. A typical sentence is short: the middle one has 10 tokens, and only 20 of the 3,000 are longer than 40 and get cut. Two sentences are just “10/10”; the tokenizer drops digits, so they become a single <unk> — a real limit of this simple tokenizer, left in on purpose.

The important number is the last one. The training sentences contain 4053 different words, but only 1721 appear twice or more. So **14.4% of the tokens in the test sentences are unknown** to the model. In the worked example, *circumstances* and *tops* both turn into <unk>. With only 2,000 training sentences, about one test word in seven is invisible. Keep this in mind for every accuracy number that follows.

2. The embedding table and masking — built, then checked

DEFINITION

An **embedding table** is a lookup table with one row per word. Each row is a short list of numbers (here 16) that stands for that word. The numbers start random and are **learned** during training, like any other weight.

A **mask** is a row of 1s and 0s that marks which positions of a padded sentence are real words (1) and which are padding (0). On a padding position the RNN's hidden state is simply carried forward unchanged, so padding cannot affect the answer.

REAL-WORLD SCENARIO

Embedding: imagine describing every dish on a menu with a few scores — how sweet, how spicy, how heavy. Two dishes with similar scores are similar dishes. An embedding does this for words, except nobody chooses what the scores mean; the model adjusts them until they are useful for the task.

Masking: a teacher marks an exam where each answer has ten lines. A student who writes three lines leaves seven blank. A fair teacher stops reading at the last written line and gives the mark from there. Without masking, the RNN would keep “reading” the blank lines and slowly forget the answer.

THE MATH

```
embedding forward :  x_t = E[id_t]                    (pick row id_t of the table)
embedding backward:  dE[id_t] += dx_t  for every t    (a word used twice gets two additions)
masked recurrence :  h_new = tanh(x_t Wxh + h_(t-1) Whh + bh)
                     h_t   = m_t * h_new + (1 - m_t) * h_(t-1)     m_t = 1 real word, 0 padding
masked backward   :  da    = dh_t * m_t * (1 - h_t^2)
                     dh_(t-1) = da Whh^T + dh_t * (1 - m_t)
```

SPECIFICATION

Gradient check	Tiny network: 12-word vocabulary, embedding 4, hidden 5. A batch of 4 sentences with lengths 5, 2, 4, 1, and one word used twice in the same sentence. Every entry of all six parameters (E, Wxh, Whh, bh, Why, by) is nudged by ±1e-5 and the change in loss compared with the gradient from backward. Done for both readouts (last, mean).
Pass criterion	Largest relative error below about 1e-6; the <pad> row of dE and the rows of unused words exactly zero.
Batch invariance	An untrained default network. One 3-word sentence is run alone (no padding), then inside a batch padded to 40 next to a 40-word sentence, with and without the mask.
Pass criterion	With the mask, the prediction must be the same as alone.

Why this test

The backward pass has two new places to go wrong. First, a word that appears twice must receive the gradient from **both** positions; the obvious NumPy line `dE[ids] += dX` silently keeps only one of them (the exercise script shows this). Second, the mask changes the recurrence, so its backward rule changes too. A numeric gradient check does not care how the code is written — it only compares against the true slope — so it catches both. Batch invariance checks something a gradient check cannot: that a sentence's answer does not depend on which other sentences share its batch.

IMPLEMENTATION

```
def embedding_forward(E, ids):
    return E[ids]
def embedding_backward(dX, ids, vocab_size):
    dE = np.zeros((vocab_size, dX.shape[-1]))
    np.add.at(dE, ids, dX)
    return dE
def rnn_forward_masked(X, mask, h0, Wxh, Whh, bh):
    m, T, _ = X.shape
    H = np.zeros((m, T, Whh.shape[0]))
```

```
    h = h0
    for t in range(T):
        h_new = np.tanh(X[:, t, :] @ Wxh + h @ Whh + bh)
        m_t = mask[:, t:t + 1]
        h = m_t * h_new + (1.0 - m_t) * h
        H[:, t, :] = h
    return H
def rnn_backward_masked(dH_direct, dh_last, X, mask, H, h0, Wxh, Whh):
    m, T, _ = X.shape
    dWxh = np.zeros_like(Wxh); dWhh = np.zeros_like(Whh); dbh = np.zeros(Whh.shape[0])
    dX = np.zeros_like(X)
    dh = dh_last
    for t in reversed(range(T)):
        dh = dh + dH_direct[:, t, :]
        m_t = mask[:, t:t + 1]
        h_prev = H[:, t - 1, :] if t > 0 else h0
        da = dh * m_t * (1.0 - H[:, t, :] ** 2)
        dWxh += X[:, t, :].T @ da / m
        dWhh += h_prev.T @ da / m
        dbh += da.sum(axis=0) / m
        dX[:, t, :] = da @ Wxh.T / m
        dh = da @ Whh.T + dh * (1.0 - m_t)
    return dWxh, dWhh, dbh, dX
```

The subtle parts: `np.add.at` adds repeated rows correctly. In the backward loop, `dh * (1.0 - m_t)` passes the gradient straight through a padding step, mirroring the forward rule that carried the state through it.

ACTUAL OUTPUT

```
 readout          E        Wxh        Whh         bh        Why         by        max
    last   1.20e-08   2.44e-09   3.28e-09   1.10e-09   1.90e-09   1.94e-09   1.20e-08
    mean   1.09e-08   7.82e-09   3.85e-08   1.93e-09   1.17e-09   1.95e-09   3.85e-08
gradient row of <pad> is exactly zero: True;  rows of the 4 unused words are exactly zero: True
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
P(positive) for the 3-word sentence:
  alone, no padding            : 0.4496053643
  in the batch, WITH mask      : 0.4496053643   |difference| = 0.00e+00
  in the batch, WITHOUT mask   : 0.4377539816   |difference| = 1.19e-02
Actual output from this lesson's run.
```

What it means

Every parameter passes for both readouts: the largest relative error is 1.20e-08 (last) and 3.85e-08 (mean). Padding and unused words receive exactly zero gradient, so training never moves their rows.

With the mask, the 3-word sentence gets the identical prediction alone and in the padded batch (difference 0.00e+00). Without it the prediction shifts by 1.19e-02. That shift looks small only because the network is untrained and its embeddings are tiny. Section 4 repeats the test on a **trained** network, where skipping the mask drops accuracy to chance.

3. The baseline: bag-of-words

DEFINITION

A **baseline** is a simple method you build first, so you have a bar that a more complicated method must clear. **Bag-of-words** describes a sentence only by **which words appear in it**. The order of the words is thrown away completely. A **logistic regression** then learns one weight per word: positive weight pushes toward “positive review”, negative weight pushes toward “negative”.

REAL-WORLD SCENARIO

Write each word of a review on a card, drop the cards into a bag, and shake. Now pull them out and count: many cards saying *great*, *love*, *delicious*? Probably a happy customer. Cards saying *worst*, *waste*, *terrible*? Probably not. Simple keyword spam filters work this way.

The weakness: “the food was good, not bad” and “the food was bad, not good” put exactly the same cards in the bag. A bag-of-words model must give both the same answer. An RNN reads the words in order, so in principle it can tell them apart. Whether it **does** is today's real question.

SPECIFICATION

Features	One column per vocabulary word (1,723 columns): 1 if the word is in the sentence, else 0.
Model	Softmax (logistic) regression, weights start at zero.
Training	Full-batch gradient descent, learning rate 2.0, 600 epochs, L2 penalty 1e-4.
Model selection	The epoch with the best validation accuracy is kept; test accuracy is read at that epoch.
Chance level	0.500. With 600 test sentences, one standard error is roughly 0.02.

Why this test

Without a baseline, an accuracy number means nothing: is 0.74 good? The baseline is chosen so that it has **everything the RNN has except word order** — same sentences, same vocabulary, same split. So if the RNN scores higher, order helped. If it does not, the RNN is not yet getting anything out of reading in sequence.

IMPLEMENTATION

```
def bag_of_words(ids, vocab_size):
    X = np.zeros((ids.shape[0], vocab_size))
    rows = np.repeat(np.arange(ids.shape[0]), ids.shape[1])
    X[rows, ids.ravel()] = 1.0
    X[:, PAD] = 0.0
    return X
def train_bow(X_tr, y_tr, X_va, y_va, lr=2.0, epochs=600, l2=1e-4):
    W = np.zeros((X_tr.shape[1], 2)); b = np.zeros(2)
    best = (-1.0, None, None, -1)
    for epoch in range(epochs):
        dlogits = (softmax(X_tr @ W + b) - one_hot(y_tr, 2)) / len(y_tr)
        W -= lr * (X_tr.T @ dlogits + l2 * W)
        b -= lr * dlogits.sum(axis=0)
        val = ((X_va @ W + b).argmax(axis=1) == y_va).mean()
        if val > best[0]:
            best = (val, W.copy(), b.copy(), epoch)
    return best
```

The subtle part: `X[rows, ids.ravel()] = 1.0` marks presence only. Position never enters the features, so this model cannot see order even in principle.

ACTUAL OUTPUT

```
best validation accuracy 0.772 at epoch 314; test accuracy there 0.817  (chance = 0.500; 600 test
    sentences, so one standard error is roughly 0.02)
most negative words: bad, poor, not, worst, slow, don't, terrible, then, waste, no
most positive words: great, love, nice, good, excellent, awesome, perfect, delicious, loved,
    enjoyed
Actual output from this lesson's run.
```

What it means

The baseline reaches **0.817** on the test sentences. The words it leans on are the ones a person would pick: *bad, poor, worst, terrible, waste* against *great, love, excellent, delicious*. Notice **not**, **don't** and **no** in the negative list: the baseline has learned that sentences containing “not” are usually complaints. It treats “not” as a bad word, not as a word that flips what follows. That shortcut comes back in Section 6.

4. Training the many-to-one RNN

DEFINITION

Many-to-one means the network takes in many inputs, one after another, and gives one answer at the end. Here the inputs are the words of a sentence and the answer is “positive” or “negative”. The RNN updates its hidden state after each word and the prediction is made from the state after the last real word.

Overfitting means the model memorizes the training examples instead of learning a rule that also works on new ones. The sign: training accuracy keeps rising while accuracy on unseen sentences stops.

REAL-WORLD SCENARIO

Many-to-one: you read a whole restaurant review, word by word, and at the end press one button: thumbs up or thumbs down. Email spam filters (whole email in, spam / not spam out) and a doctor reading a full list of symptoms to give one diagnosis have the same shape.

Overfitting: a student who memorizes the answers to last year's exam scores 100% on last year's paper and poorly on this year's. The **validation set** is the practice exam used to decide when to stop studying; the **test set** is the real exam, opened only once at the end.

SPECIFICATION

Network	Embedding 16 → masked vanilla RNN, hidden 16 (tanh) → 2-class softmax, reading the last real hidden state. 28,130 weights, of which 27,568 are the embedding table.
Training	SGD, learning rate 0.1, batch 32, 40 epochs, gradient clipping at global norm 1.0 (Day 60), seed 0. Each batch is padded only to its own longest sentence.
Tracked every epoch	Mean training loss; accuracy on training, validation and test sentences; fraction of steps that were clipped.
Model selection	The epoch with the best **validation** accuracy. Test accuracy is recorded but never used to choose.
Extra check	The chosen network is evaluated once more with the mask switched off.

Why this test

This is the first time every part built this week runs together on real data. Accuracy is recorded **during** training, not only at the end, because Day 60 showed that an end-of-training snapshot can hide what really happened. The no-mask check asks whether Section 2's masking matters in practice or only in a unit test.

IMPLEMENTATION

```
def forward(self, ids, lens, use_mask=True):
    m, T = ids.shape
    self.ids, self.lens = ids, lens
    self.mask = (np.arange(T)[None, :] < lens[:, None]).astype(float) if use_mask else np.ones((m,
        T))
    self.h0 = np.zeros((m, self.hidden_dim))
    self.X = embedding_forward(self.E, ids)
    self.H = rnn_forward_masked(self.X, self.mask, self.h0, self.Wxh, self.Whh, self.bh)
    if self.readout == "last":
        self.feat = self.H[:, -1, :]
    else:
        self.feat = (self.H * self.mask[:, :, None]).sum(axis=1) / lens[:, None]
    self.probs = softmax(self.feat @ self.Why + self.by)
```

```
    return self.probs
def gradients(self, y):
    m, T = self.ids.shape
    dlogits = self.probs - one_hot(y, 2)
    grads = dict(Why=self.feat.T @ dlogits / m, by=dlogits.sum(axis=0) / m)
    dfeat = dlogits @ self.Why.T
    if self.readout == "last":
        dH_direct, dh_last = np.zeros_like(self.H), dfeat
    else:
        dH_direct = dfeat[:, None, :] * self.mask[:, :, None] / self.lens[:, None, None]
        dh_last = np.zeros_like(dfeat)
    grads["Wxh"], grads["Whh"], grads["bh"], dX = rnn_backward_masked(
        dH_direct, dh_last, self.X, self.mask, self.H, self.h0, self.Wxh, self.Whh)
    grads["E"] = embedding_backward(dX, self.ids, self.vocab_size)
    return grads
def train_step(self, ids, lens, y):
    loss = cross_entropy_loss(self.forward(ids, lens), one_hot(y, 2))
    grads, _, scale = clip_by_global_norm(self.gradients(y), self.clip)
    for name, g in grads.items():
        getattr(self, name)[...] -= self.lr * g
    return loss, scale < 1.0
```

The subtle part: because padding steps carry the state forward, `self.H[:, -1, :]` is the state after each sentence's last *real* word, even though the sentences in the batch have different lengths.

IMPLEMENTATION — the training loop inside fit()

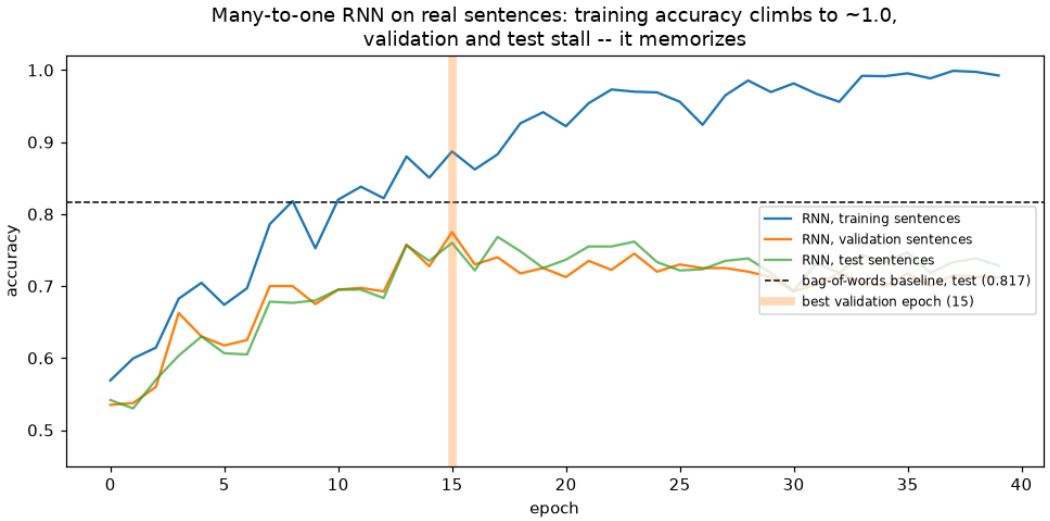
```
for epoch in range(epochs):
    if order == "original":
        src = ids_tr
    elif order == "shuffled once":
        src = shuffle_within(ids_tr, len_tr, seed=1000)
    else:
        src = shuffle_within(ids_tr, len_tr, seed=1000 + epoch)
    perm = rng.permutation(len(y_tr))
    losses, clips = [], []
    for start in range(0, len(perm), batch_size):
        idx = perm[start:start + batch_size]
        T_batch = len_tr[idx].max()
        loss, was_clipped = net.train_step(src[idx][:, :T_batch], len_tr[idx], y_tr[idx])
        losses.append(loss); clips.append(was_clipped)
    hist["loss"].append(np.mean(losses)); hist["clipped"].append(np.mean(clips))
    hist["train"].append(net.accuracy(ids_tr, len_tr, y_tr))
    hist["val"].append(net.accuracy(ids_va, len_va, y_va))
    hist["test"].append(net.accuracy(ids_te, len_te, y_te))
    if hist["val"][-1] > best_val:
        best_val, best_net, best_epoch = hist["val"][-1], copy.deepcopy(net), epoch
```

ACTUAL OUTPUT

```
 epoch  train loss  train acc  val acc  test acc  steps clipped
     0      0.6899      0.569    0.535     0.542           4.8%
     3      0.6418      0.682    0.662     0.603          19.0%
     6      0.5548      0.697    0.625     0.605          71.4%
     9      0.4690      0.752    0.675     0.680          84.1%
    12      0.3766      0.822    0.693     0.683          93.7%
    15      0.3277      0.887    0.775     0.760          93.7%
    18      0.2597      0.926    0.718     0.748          95.2%
    21      0.2032      0.954    0.735     0.755          93.7%
    24      0.1443      0.969    0.720     0.733          85.7%
    27      0.1195      0.965    0.725     0.735          82.5%
    30      0.0760      0.982    0.693     0.693          69.8%
    33      0.0433      0.992    0.705     0.743          49.2%
    36      0.0263      0.989    0.700     0.718          30.2%
    39      0.0116      0.993    0.713     0.728          12.7%
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
best validation accuracy 0.775 at epoch 15: train 0.887, test 0.760
at the end (epoch 39): train 0.993, validation 0.713, test 0.728
bag-of-words baseline on the same split: validation 0.772, test 0.817
same best-epoch network, but reading the state after the padding instead (no mask): test 0.518
Actual output from this lesson's run.
```



Training accuracy (blue) keeps climbing; validation (orange) and test (green) level off. The dashed line is the bag-of-words baseline.

What it means

It learns. From chance, the RNN reaches 0.775 on validation at epoch 15, where its test accuracy is 0.760. Real sentences, real labels, every piece written from scratch.

Then it memorizes. By the last epoch training accuracy is 0.993 while validation has slid to 0.713 and test to 0.728. Almost all of the network's weights are embedding numbers, and 2,000 sentences are few enough to learn by heart.

It does not beat the baseline. Bag-of-words scores 0.817 on the same test sentences. One seed is one sample, so Section 5 repeats this over five seeds before drawing any conclusion.

Masking matters in practice. The same trained network, read after the padding instead of after the last real word, scores 0.518 — chance. Up to 39 padding steps wash the sentence out of the hidden state, the forgetting Day 57 measured.

Clipping is busy. In the middle of training more than 90% of steps are clipped, so for most of this run the size of each step is set by the clip threshold, not by the raw gradient.

5. Which state to read: the last one, or the average of all?

DEFINITION

The **readout** is the rule that turns the RNN's many hidden states into one prediction. **Last** reads only the hidden state after the final word. **Mean** (also called mean pooling) averages the hidden states after every real word and predicts from that average.

REAL-WORLD SCENARIO

You watch a film and must rate it. **Last** is rating it only by how you feel as the credits roll: a weak ending can wipe out a great first hour. **Mean** is noting how you feel after every scene and averaging the notes: no single scene, early or late, can be forgotten.

For a review like “Terrible service, and the waiter who finally came to our table told us the kitchen had closed”, the key word is at the very start. A last-state reader has to carry “terrible” in memory across fifteen more words.

SPECIFICATION

Compared	Readout = last vs. mean, each at learning rate 0.1 and 0.3, clipping at 1.0. One extra row: last, lr 0.1, **no** clipping.
Repeats	Five seeds (0–4) per setting. The seed changes the starting weights and the batch order.
Reported	Mean validation and test accuracy at each run's best validation epoch; spread across seeds; training accuracy at the end.
By length	Test sentences grouped by number of tokens (1–6, 7–12, 13–20, 21–40), using each readout's validation-chosen learning rate.
Everything else	Same as Section 4.

Why this test

Days 57–59 showed that a vanilla RNN has trouble keeping early information until the end of a sequence, and trouble sending gradient back to early steps. The last-state readout depends on both. The mean readout gives every word's hidden state a direct line to the output, in both directions. If long-range memory is what limits the RNN here, mean should do better, and the gap should be largest on long sentences.

IMPLEMENTATION

```
if self.readout == "last":
    self.feat = self.H[:, -1, :]
else:
    self.feat = (self.H * self.mask[:, :, None]).sum(axis=1) / lens[:, None]
self.probs = softmax(self.feat @ self.Why + self.by)
return self.probs
# ... and in gradients(), where the gradient enters the backward pass:
if self.readout == "last":
    dH_direct, dh_last = np.zeros_like(self.H), dfeat
else:
    dH_direct = dfeat[:, None, :] * self.mask[:, :, None] / self.lens[:, None, None]
    dh_last = np.zeros_like(dfeat)
```

The subtle part: with the mean readout, each real hidden state receives its own share (1/length) of the output gradient directly, instead of everything arriving through the last state.

ACTUAL OUTPUT

```
 readout    lr  clip  val mean  test mean  test std  test min  test max  final train
    last   0.1   1.0     0.737      0.737     0.035     0.677     0.773        0.997
    last   0.3   1.0     0.686      0.688     0.011     0.672     0.698        1.000
    mean   0.1   1.0     0.740      0.765     0.015     0.743     0.788        0.928
    mean   0.3   1.0     0.766      0.797     0.013     0.777     0.808        0.988
    last   0.1  None     0.708      0.712     0.028     0.677     0.752        1.000
bag-of-words baseline     0.772      0.817
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
learning rate chosen on validation: last -> 0.1, mean -> 0.3
test accuracy by sentence length (mean over the five seeds):
    length  sentences    last    mean  bag-of-words
       1-6        196   0.817   0.854         0.862
      7-12        182   0.710   0.768         0.802
```

```
     13-20        148   0.726   0.780         0.811
     21-40         74   0.614   0.754         0.743
Actual output from this lesson's run.
```

What it means

Mean beats last, and is steadier. Averaged over five seeds: last 0.737 (individual runs from 0.677 to 0.773), mean 0.797 (from 0.777 to 0.808). The last-state reader's result depends heavily on the seed; the mean reader's barely does.

Neither beats the baseline. Mean, at 0.797, is within about one standard error of bag-of-words (0.817) on a 600-sentence test set, so those two cannot be told apart here. Last is clearly behind.

Long sentences. All three methods lose accuracy as sentences get longer. The last-state reader loses the most: 0.817 on 1–6 tokens down to 0.614 on 21–40, where mean holds 0.754. That fits the long-range-memory story from Days 57–59. Treat it as a hint and not a proof: the longest group has only 74 sentences.

Clipping. Without it the last-state reader averages 0.712 against 0.737 with it. The seed-to-seed spread is about 0.03, so this run shows **no clear effect** of clipping either way. Unlike Day 60's task, nothing here collapses without it.

6. Does the RNN use word order?

DEFINITION

Word order is the one thing an RNN can see that bag-of-words cannot. An **ablation** is an experiment where you take one thing away on purpose and watch what changes. If nothing gets worse, that thing was not being used. Here the thing taken away is the order of the words.

REAL-WORLD SCENARIO

“Dog bites man” and “man bites dog” use the same three words and mean very different things. “Not bad at all” is praise; “bad, not good at all” is a complaint. Order carries meaning, and a reader who ignores it will sometimes be badly wrong.

Ablation in daily life: the lights keep tripping, so you unplug one appliance at a time. If the lights still trip with the kettle unplugged, the kettle was not the cause. Below, word order is “unplugged” by shuffling the words, to see whether the RNN was relying on it.

SPECIFICATION

Network	The last-state RNN from Section 5 (lr 0.1, clip 1.0), five seeds.
(a) Shuffle at test time	The words inside every **test** sentence are put in random order (five different shuffles). Measured: accuracy on the shuffled sentences, and the fraction of predictions that change.
(b) Shuffle at training time	Three training conditions: **original** order; **shuffled once** (each training sentence scrambled one fixed way, reused every epoch); **reshuffled each epoch** (a new scramble every epoch). All are tested on normal, unshuffled sentences.
(c) Negation probes	Six sentence pairs written by hand for this lesson (not from the dataset), each a phrase and its “not” version. Measured: P(positive) from the seed-0 RNN and from the baseline.
If order is being used	(a) accuracy should fall; (b) the shuffled conditions should score lower than original; (c) “not bad” should come out positive.

Why this test

Sections 4 and 5 showed the RNN does not beat a model that cannot see order. That leaves two possibilities: the RNN uses order but loses elsewhere, or it is not getting anything useful from order at all. These three tests separate them. “Shuffled once” is the key control: it removes real word order but gives the network the same amount of data, in the same form, as the original run. “Reshuffled each epoch” also removes order, but in addition shows the network a different version of each sentence every time.

IMPLEMENTATION

```
def shuffle_within(ids, lens, seed):
    rng = np.random.RandomState(seed)
    out = ids.copy()
    for row in range(len(lens)):
        out[row, :lens[row]] = rng.permutation(ids[row, :lens[row]])
    return out
# inside fit(): which version of the training sentences this epoch uses
if order == "original":
    src = ids_tr
elif order == "shuffled once":
    src = shuffle_within(ids_tr, len_tr, seed=1000)
else:
    src = shuffle_within(ids_tr, len_tr, seed=1000 + epoch)
# inside demo_word_order(): the test-time shuffle
p0 = net.predict(ids_te, len_te)
ps = [net.predict(shuffle_within(ids_te, len_te, s), len_te) for s in range(5)]
val.append(h["val"][be]); test.append((p0 == y_te).mean())
shuf.append(np.mean([(p == y_te).mean() for p in ps]))
flip.append(np.mean([(p != p0).mean() for p in ps]))
```

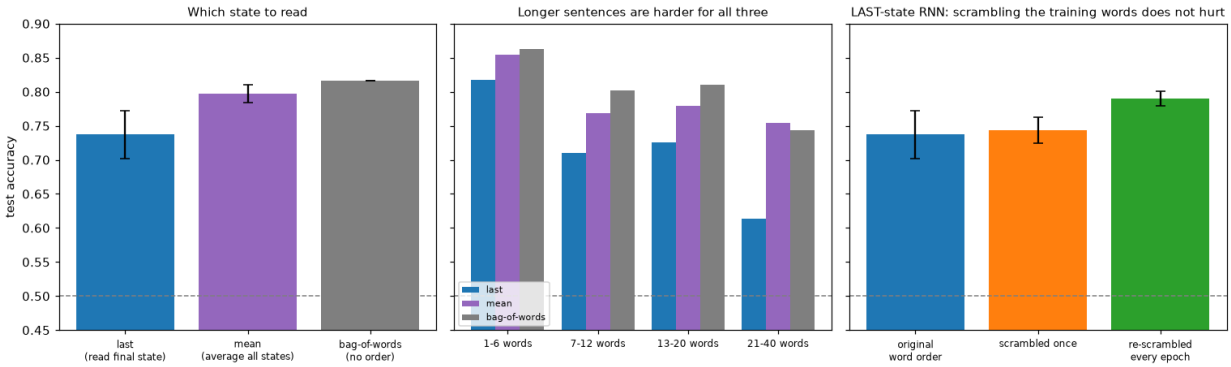
The subtle part: only the positions `[:lens[row]]` are shuffled, so padding stays at the end and the mask is still correct.

ACTUAL OUTPUT

```
             trained on     val    test  test std  test, words shuffled  predictions flipped
               original   0.737   0.737     0.035                 0.726                20.5%
          shuffled once   0.730   0.744     0.020                 0.753                16.3%
   reshuffled each epoch   0.764   0.790     0.011                 0.794                 5.8%
   bag-of-words baseline   0.772   0.817                           0.817                 0.0%
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
negation probes (hand-written for this lesson, not from the dataset) -- P(positive):
sentence                          RNN    BoW   negated sentence                  RNN    BoW
good                            0.924  0.855   not good                        0.070  0.373
bad                             0.012  0.033   not bad                         0.001  0.003
i love this phone               0.902  0.871   i do not love this phone        0.066  0.259
the food was great              0.762  0.940   the food was not great          0.657  0.614
would recommend                 0.532  0.580   would not recommend             0.026  0.122
this movie is terrible          0.071  0.172   this movie is not terrible      0.007  0.020
Actual output from this lesson's run.
```



Left: test accuracy by readout (bars show seed-to-seed spread). Middle: accuracy by sentence length. Right: the last-state RNN trained on original, scrambled-once and re-scrambled sentences.

What it means

(a) The RNN's answers depend on order, but not usefully. Shuffling the test words changes 20.5% of its predictions, so it is not literally a bag-of-words. Yet accuracy barely moves: 0.737 on normal sentences, 0.726 on shuffled ones, a difference smaller than the seed-to-seed spread. The flips roughly cancel out. Its sensitivity to order is noise, not knowledge.

(b) Taking order away during training costs nothing. Trained on sentences scrambled once, the RNN scores 0.744 against 0.737 for the original order — no loss. Whatever the network learned from real word order, it did not help on new sentences.

Re-scrambling every epoch helps. That run reaches 0.790, with a much smaller spread (0.011), and only 5.8% of its predictions change under a test-time shuffle. Since “shuffled once” did not help, the gain does not come from removing order as such. It comes from showing the network 40 different versions of each sentence: it can no longer memorize exact sequences, and is pushed toward answers that do not depend on order — which is what works on this data.

(c) “Not” is treated as a bad word, not as a switch. “good” to “not good” drops from 0.924 to 0.070, which looks right. But “bad” to “not bad” goes from 0.012 to 0.001, and “this movie is not terrible” gets 0.007: more negative, when a reader would say more positive. The baseline does the same. Both learned that “not” usually appears in complaints. These six pairs are hand-written examples and not a measurement, but they agree with (a) and (b).

READ THIS CAREFULLY

What this does and does not show. It does **not** show that word order is unimportant for sentiment, or that RNNs cannot learn it. It shows that **this** vanilla RNN, trained from scratch on 2,000 sentences with about one test word in seven unknown, did not learn to use order in a way that helps. Section 7 lists what usually changes that.

7. What this means in practice, and what comes next

DEFINITION

A **negative result** is a careful experiment whose answer is “no, this did not help”. It is still a result: it tells you where not to spend effort, and what to try instead.

REAL-WORLD SCENARIO

A café buys an expensive espresso machine and finds that, for the twenty cups it sells a day, the old filter pot was just as good and easier to run. The machine is not bad; the café is not yet big enough to need it. Small data and a simple task are the twenty cups a day.

- **Always build the simple baseline first.** Here it took a few lines, trained in seconds, and won. Without it, 0.74 from a from-scratch RNN would have looked like success.
- **What usually closes the gap** (none of it done today): far more training text; embeddings trained beforehand on a large body of text, so that words outside the 2,000 sentences are not unknown; and a cell built to hold information across many steps — the LSTM, Day 63.
- **The machinery is ready.** Tokens, ids, padding, an embedding table, masking, a many-to-one readout and clipping all exist and are checked. Those parts are the same ones a larger model uses.
- **Day 62** changes the output side: many-to-many, one label for every word instead of one per sentence. There, position is the whole task, so an order-blind model cannot be used as a shortcut.

8. Interview prep

What does “many-to-one” mean for an RNN?

The network reads a sequence of inputs, one per timestep, and produces a single output for the whole sequence. Sentiment classification is the standard example: many words in, one label out. The loss is computed once per sequence, and its gradient flows back through every timestep.

What is an embedding layer, and how do you backpropagate through it?

A lookup table with one learned vector per word. Forward selects rows by word id. Backward adds each position's input gradient into the row of the word at that position; a word that appears several times gets several additions. In NumPy that needs `np.add.at`, because `dE[ids] += dX` applies repeated indices only once.

Why do you need padding and masking?

Padding makes sentences of different lengths fit in one array. Masking stops the padding from changing the result. Today, a trained network read without the mask fell from 0.760 to 0.518 test accuracy, because the RNN kept updating its state over the padding and forgot the sentence.

Why build the vocabulary from the training set only?

The test set stands in for future, unseen text. If test words are used to build the vocabulary, the model is given information it would not have in real use, and the test score becomes too optimistic. Unseen words map to <unk>; here that was 14.4% of test tokens.

Last hidden state or mean pooling — which would you use?

With a vanilla RNN and longer inputs, mean pooling is usually the safer choice: every timestep gets a direct path to the output and a direct gradient. Here it scored 0.797 against 0.737 and varied far less between seeds. With an LSTM or with short inputs the difference is often small.

Your RNN scores below a bag-of-words model. What do you check?

Whether it is using order at all: shuffle the words at test time and see if accuracy drops; train on shuffled sentences and see if it gets worse. Then look at data size, unknown-word rate, overfitting (training vs. validation curves) and whether pre-trained embeddings or a gated cell are available.

How did you choose the epoch and the learning rate without cheating?

On a separate validation set. Test accuracy was recorded every epoch for the record but never used to make a choice, so the reported test numbers are honest estimates for new sentences.

9. Key takeaways

- Text reaches a network as integers: tokenize, map to ids with a vocabulary built on training data, pad to a common length.
- An embedding is a learned lookup table. Its backward pass is a scatter-add; padding and unused words get zero gradient.
- Masking carries the hidden state through padding. It is exact (difference 0.00e+00), and without it a trained model fell to chance.
- On 2,000 real sentences the RNN learned, then memorized (training 0.993). Test accuracy: bag-of-words 0.817, RNN last state 0.737, RNN mean of states 0.797. The RNN did not win.
- Shuffling the training words did not hurt the RNN; re-shuffling every epoch helped (0.790). It was not getting useful signal from word order.
- Both models treat “not” as a negative word rather than as something that flips the meaning.
- Build the baseline first, and test whether the fancy model is using the thing that makes it fancy.